Gap-Aware Retrieval (GAR)

A Gap-Aware Reranking Algorithm for Iterative Retrieval-Augmented Generation

Abstract

Retrieval systems commonly rank documents according to their relevance to the original query. This can be insufficient in iterative information-seeking tasks, where the system already has some evidence and needs to retrieve only the information that is still missing. This project proposes **Gap-Aware Retrieval (GAR)**, a reranking algorithm that reorders a first-stage candidate set using three signals: relevance to the question, coverage of the missing information, and novelty with respect to previously collected evidence.

GAR is evaluated inside the existing MIGRES framework, which identifies missing information during iterative retrieval. MIGRES is the experimental framework; a first-stage retriever performs retrieval from the corpus, and GAR is the proposed reranking component applied on top of that retriever's candidates. Which first-stage retriever is used is itself selected empirically (Section 2) rather than fixed in advance; BM25 is one of the candidates compared for that role. The surrounding MIGRES components are kept fixed as far as the original implementation permits. GAR is compared against no reranking and against ten established reranking algorithms — cross-encoder reranking (MonoBERT), MonoT5, pairwise reranking (DuoBERT), ColBERTv2 late-interaction reranking, classical learning-to-rank (LambdaMART), neural learning-to-rank (RankT5), Maximal Marginal Relevance (MMR), coverage-driven reranking (AIR), relevant-information-gain reranking (Dartboard/RIG), and LLM-based listwise reranking (RankGPT) — on the datasets used in the MIGRES evaluation: WikiHop, HotpotQA, MuSiQue, Natural Questions, TriviaQA, and StrategyQA.

The study evaluates retrieval quality, downstream answer quality, ablations, efficiency, and statistical significance. GAR has no trainable parameters in the proposed implementation. Its scoring hyperparameters are selected on a held-out validation set using Bayesian optimization and then frozen before test evaluation. The formulation is motivated by prior work on missing-information-guided retrieval, iterative evidence coverage, relevant information gain, sentence embeddings, and efficient nearest-neighbor search [Wang et al., 2025, Yadav et al., 2020, Pickett et al., 2024, Reimers and Gurevych, 2019, Johnson et al., 2017].

1 Introduction

Retrieval-Augmented Generation (RAG) systems use external documents to provide a language model with information that may not be available in its parameters. A retrieval algorithm therefore has a simple but important job: given a question and a document collection, return a small set of useful documents.

A common retrieval objective is query–document relevance:

$$q + \mathcal{D} \longrightarrow d_1, d_2, \ldots, d_k.$$

This works well when the question can be answered from one retrieval step. It becomes less direct when the system retrieves information iteratively. After one step, some parts of the information need may already be satisfied. The next retrieval step should focus on the remaining gap rather than repeatedly retrieving documents about the same topic.

MIGRES provides a useful setting for this problem. MIGRES is a Missing Information Guided Retrieve-Extraction-Solving framework. It identifies missing information after evidence is collected

and uses that missing information to guide subsequent retrieval [Wang et al., 2025]. In this project, we do not modify the MIGRES reasoning, extraction, or solving framework. Instead, we replace its retrieval component with GAR.

The central research question is:

$$\boxed{\text{Does explicitly modeling the current information gap improve retrieval?}}$$

GAR addresses this by scoring a candidate document using:

$$\boxed{S(d \mid q, E_t, G_t) = \alpha R(d, q) + \beta C(d, G_t) + \gamma N(d, E_t)}$$

where R measures relevance, C measures gap coverage, and N measures evidence novelty.

The contribution of this project is the proposed GAR reranking formulation, applied on top of a first-stage retriever's candidates, and its controlled empirical evaluation. Which first-stage retriever is used is selected empirically (Section 2) rather than assumed; GAR's own comparison against reranking baselines (Section 3, Section 8) is run on top of whichever retriever that selection identifies. The individual ideas used to construct GAR are grounded in previous work; the project does not claim that semantic relevance, information coverage, or novelty are individually new.

1.1 What is MIGRES and How Does It Work?

MIGRES stands for *Missing Information Guided Retrieve-Extraction-Solving*. It is an iterative framework designed for questions that require multiple pieces of information. Instead of retrieving documents only once, MIGRES repeatedly checks what information is already available, identifies what is still missing, retrieves information for that gap, and uses the retrieved information to continue solving the question [Wang et al., 2025].

The basic idea is:

$$\boxed{\begin{array}{c}\text{Question} \rightarrow \text{Find Missing Information} \rightarrow \text{Retrieve} \rightarrow \text{Extract} \\ \rightarrow \text{Update Information} \rightarrow \text{Repeat if Needed}\end{array}}$$

In simple terms, MIGRES asks three questions during the process:

1. **What do we already know?** It keeps the evidence collected so far.
2. **What information is still missing?** It identifies the missing information needed to answer the question.
3. **What should we retrieve next?** It uses the missing information to guide the next retrieval step.

For example, suppose a question requires two facts, A and B. After the first retrieval, MIGRES may obtain A but determine that B is still missing. It then focuses the next retrieval step on B:

$$\text{Question} \rightarrow \text{retrieve } A \rightarrow \text{missing } B \rightarrow \text{retrieve } B \rightarrow \text{answer.}$$

MIGRES therefore maintains an iterative state consisting of the original question q, previously collected evidence E_t, and the current missing-information requirements G_t. These are provided to the retrieval component at each iteration:

$$\boxed{(q, E_t, G_t, \mathcal{D})}$$

where $\mathcal{D}$ is the searchable document corpus.

In our work, **MIGRES is the surrounding framework, not our contribution**. We use its iterative missing-information and evidence-generation process, while replacing its retrieval component with our proposed GAR algorithm. MIGRES determines *what information is missing*; GAR determines *which documents are most useful for obtaining that information.*

Thus, the division of responsibility is:

$$\underbrace{\text{MIGRES}}_{\text{finds what is missing}} \rightarrow \underbrace{\text{GAR}}_{\text{reranks useful documents}} \rightarrow \underbrace{\text{MIGRES}}_{\text{extracts and updates evidence}}$$

This allows us to test whether a gap-aware reranking algorithm improves the ranking stage while keeping the rest of the MIGRES framework, including first-stage retrieval, unchanged.

1.2 What We Use from MIGRES

We use MIGRES as the surrounding experimental framework. MIGRES produces the iterative state:

$$(q, E_t, G_t, \mathcal{D}),$$

where:

- q is the original question;
- E_t is evidence collected before retrieval iteration t;
- G_t is the current missing-information state;
- $\mathcal{D}$ is the searchable document corpus.

This state is used in two stages. First, a first-stage retriever performs retrieval from $\mathcal{D}$ using q and each requirement in G_t, producing a candidate set $\mathcal{C}_t$. Second, GAR reranks $\mathcal{C}_t$ using q, E_t, and G_t:

$$T_t = \text{TopK}_{d \in \mathcal{C}_t} S(d \mid q, E_t, G_t).$$

MIGRES then continues its normal filtering, extraction, gap-analysis, and solving process.

Throughout this project we write FSR (First-Stage Retriever) for whichever concrete algorithm performs this first stage. Which algorithm FSR refers to is not fixed in advance: Section 2 compares BM25 against several alternative first-stage retrievers, and the best-performing one on validation data is selected and then held fixed for the remainder of the project, so that the GAR-vs-baselines comparison in Section 3 and Section 8 isolates the effect of the reranking method rather than the first-stage retriever.

We therefore compare:

$$\boxed{\text{MIGRES+FSR (no rerank)}} \quad \text{vs.} \quad \boxed{\text{MIGRES+FSR+GAR}}.$$

We additionally compare GAR with ten other established reranking methods, each applied on the same FSR candidate set $\mathcal{C}_t$ (Section 3).

1.3 What We Do Not Change

The following MIGRES components should remain fixed wherever the original implementation permits:

- question processing;
- information-gap generation;
- query-generation logic;
- knowledge filtering;
- information extraction;
- answer-generation/solving model;
- stopping conditions;
- prompts and model settings;
- dataset and corpus.

The main experimental variable is therefore the retrieval algorithm.

2 Information Retrieval Algorithms

2.1 What is an Information Retrieval Algorithm?

An *Information Retrieval (IR) algorithm* is a method used to search a collection of documents and rank the documents according to how useful they are for a given information need. In our setting, the information need is represented by a question or retrieval query q, and the IR algorithm produces an ordered list of documents:

$$\boxed{(q, \mathcal{D}) \rightarrow d_1, d_2, \ldots, d_k}$$

where $\mathcal{D}$ is the document collection and $d_1, \ldots, d_k$ are the top-k retrieved documents.

Different IR algorithms use different signals to decide which documents should be ranked highly. For example, BM25 primarily uses lexical term-matching, DPR and Contriever use dense vector representations, ColBERTv2 uses token-level late interaction, and SPLADE uses sparse neural representations [Robertson and Zaragoza, 2009, Karpukhin et al., 2020, Izacard et al., 2022, Santhanam et al., 2022, Formal et al., 2021].

In a conventional retrieval system, the main objective is:

$$\boxed{\text{Find documents that are relevant to the query.}}$$

However, in an iterative system such as MIGRES, relevance to the original question may not be sufficient. Some information may already have been retrieved, while other information is still missing. This motivates GAR, which additionally considers the current information gap and the evidence already collected.

Thus, the difference between the first-stage retrieval algorithms and GAR is not that GAR is simply another embedding model. GAR is a reranking method whose scoring function explicitly incorporates the current information state, applied to the candidates the first-stage retriever already retrieved:

$$\boxed{\text{Question} + \text{Information Gap} + \text{Existing Evidence} \rightarrow \text{Document Ranking.}}$$

2.2 Why Compare Retrieval Algorithms?

These five methods represent the established landscape of first-stage retrieval, and comparing them is itself a predefined experiment in this project: which one performs best on validation data is selected empirically, not assumed in advance. GAR itself is not one of these five: it is a reranking algorithm applied on top of whichever retriever is selected here, and its own comparison against established reranking algorithms is introduced in Section 3. The primary first-stage retrieval systems considered here are:

BM25, DPR, Contriever, ColBERTv2, SPLADE

These five methods are used through their published or official implementations where possible. Whichever of them achieves the best first-stage retrieval quality on D_{val} (Section 6) is selected as FSR and held fixed for the rest of the project — feeding both GAR and the reranking baselines in Section 3 with an identical candidate set. Until that selection is made, BM25 is used only as the running illustrative example in the rest of this section and in the worked examples below, because it is the simplest and most standard first-stage method to describe; it is not thereby assumed to be the winner.

2.3 BM25

BM25 is a sparse probabilistic retrieval method based mainly on query-term frequency, inverse document frequency, and document-length normalization [Robertson and Zaragoza, 2009].

A common form is:

$$S_{\text{BM25}}(q,d) = \sum_{t \in q} IDF(t) \frac{f(t,d)(k_1+1)}{f(t,d) + k_1\left(1 - b + b\frac{|d|}{avgdl}\right)}.$$

Here $f(t,d)$ is the frequency of term t in document d, $|d|$ is document length, $avgdl$ is average document length, and k_1, b are BM25 parameters. A common inverse-document-frequency choice is $IDF(t) = \ln\left(1 + \frac{N - n_t + 0.5}{n_t + 0.5}\right)$, where N is the number of documents and n_t the number containing t; implementations differ in this and in tokenization, so the exact implementation and parameters must be recorded.

BM25 is especially important to include in this comparison because it is the retrieval baseline used in the original MIGRES paper, which makes it the natural reference point — but this project treats the choice of first-stage retriever as an open empirical question to be settled by the comparison above, rather than adopting the original MIGRES choice unchanged.

2.4 DPR

Dense Passage Retrieval (DPR) uses separate dense encoders for questions and passages and retrieves documents according to their vector similarity [Karpukhin et al., 2020].

The similarity is the dot product of the two encoders' outputs:

$$S_{\text{DPR}}(q,d) = E_Q(q)^T E_P(d),$$

where E_Q and E_P are two independently parameterized BERT encoders and the [CLS] representation is used as the dense vector.

DPR provides a supervised dense-retrieval baseline.

2.5 Contriever

Contriever is an unsupervised dense retrieval method based on contrastive learning [Izacard et al., 2022]. Queries and documents are encoded independently by the same encoder f_θ, whose output is the average of the last-layer hidden states, and the relevance score is their dot product:

$$S_{\text{Contriever}}(q, d) = \langle f_\theta(q), f_\theta(d) \rangle.$$

Contriever provides a dense-retrieval baseline with a different training approach from DPR.

2.6 ColBERTv2

ColBERTv2 uses late interaction. Instead of representing a complete document with only one vector, it keeps token-level representations and computes efficient token-level interactions [Santhanam et al., 2022].

The late-interaction (MaxSim) score is:

$$S_{\text{ColBERT}}(q, d) = \sum_{i \in q} \max_{j \in d} sim(\mathbf{q}_i, \mathbf{d}_j),$$

where $\mathbf{q}_i, \mathbf{d}_j$ are normalized token embeddings and sim is their dot product (equivalently, cosine similarity). ColBERTv2 keeps this scoring rule and changes how the token embeddings are compressed and trained.

This gives a useful comparison with both single-vector dense retrieval and sparse retrieval.

2.7 SPLADE

SPLADE is a neural sparse retrieval method that produces sparse vocabulary representations for queries and documents [Formal et al., 2021].

For an input $t = (t_1, \ldots, t_N)$ with BERT embeddings $(h_1, \ldots, h_N)$, let $w_{ij} = \text{transform}(h_i)^T E_j + b_j$ be the importance of vocabulary token j for input token i, computed from the masked-language-model head. SPLADE aggregates it with a log-saturation:

$$w_j = \sum_{i \in t} \log\big(1 + \text{ReLU}(w_{ij})\big), \qquad \mathbf{s} = (w_j)_{j \in V},$$

and the retrieval score is the dot product of the query and document representations:

$$S_{\text{SPLADE}}(q, d) = \mathbf{s}_q^T \mathbf{s}_d.$$

SPLADE provides a neural sparse baseline.

2.8 Baseline Comparison

System	Retrieval type	Role
MIGRES+BM25	Sparse lexical	Candidate first-stage retriever
MIGRES+DPR	Dense bi-encoder	Candidate first-stage retriever
MIGRES+Contriever	Unsupervised dense	Candidate first-stage retriever
MIGRES+ColBERTv2	Late interaction	Candidate first-stage retriever
MIGRES+SPLADE	Neural sparse	Candidate first-stage retriever

All five are evaluated on D_{val} under the same retrieval-quality metrics (Section 8), and established implementations and published checkpoints are used where possible; their exact checkpoint, index, and retrieval parameters must be recorded. The winner of this comparison becomes FSR, held fixed for the rest of the project. All baselines should use established implementations and published checkpoints where possible. GAR does not appear in this table: it is not a first-stage retriever, but a reranking algorithm applied on top of FSR's output. GAR's own comparison, against ten established reranking algorithms, is given below.

3 Information Reranking Algorithms

3.1 What is Reranking?

A *reranking algorithm* is the second stage of a two-stage retrieval pipeline. A first-stage retriever (denoted FSR; see Section 2) narrows a large corpus $\mathcal{D}$ down to a small candidate set $\mathcal{C}_t$ using a cheap, high-recall method. A reranker then re-scores and reorders only that candidate set using a signal that would be too expensive or too fine-grained to run over the full corpus $\mathcal{D}$ directly:

$$\boxed{(q, \mathcal{C}_t) \to d_1, d_2, \ldots, d_k, \qquad \mathcal{C}_t \subseteq \mathcal{D}, \quad |\mathcal{C}_t| \ll |\mathcal{D}|}$$

Reranking never introduces documents that the first-stage retriever missed; it can only reorder what was already retrieved. This is the key structural difference from a full retrieval algorithm, which must search $\mathcal{D}$ itself.

Example. Suppose the first-stage retriever returns 100 passages for the question "What year was the treaty signed?" because they share lexical terms with the query ("treaty", "signed"). Many of these 100 passages discuss the treaty's causes, its signatories, or its consequences without ever stating the year. A reranker re-examines only these 100 candidates and reorders them so that passages actually containing a year near the word "treaty" are pushed to the top, even if their raw first-stage score was not the highest.

3.2 Why Compare Reranking Algorithms?

GAR reranks the candidate set $\mathcal{C}_t$ returned by FSR, so the correct comparison is against established reranking algorithms operating on the same candidate set, rather than against full first-stage retrievers. The primary systems are:

MonoBERT, MonoT5, DuoBERT, ColBERTv2 (reranker), LambdaMART RankT5, MMR, AIR, Dartboard / RIG, RankGPT, GAR

Each is applied to the identical FSR candidate set $\mathcal{C}_t$, so only the reranking method differs across systems.

3.3 Cross-Encoder Reranking (MonoBERT)

MonoBERT is a pointwise reranker. The query is fed to BERT as the first segment and the candidate passage as the second; the [CLS] vector is passed to a single-layer network that outputs the probability that the candidate is relevant, and every candidate is scored independently [Nogueira and Cho, 2019, Nogueira et al., 2019].

$$S_{\text{MonoBERT}}(q, d) = s_d = P(\text{relevant} \mid q, d),$$

trained with the cross-entropy loss $-\sum_{j\in J_{\text{pos}}} \log s_j - \sum_{j\in J_{\text{neg}}} \log(1-s_j)$. Because the question and document are encoded jointly, MonoBERT captures fine-grained term interactions that bi-encoder retrieval methods cannot, at the cost of requiring a full forward pass per candidate — practical only because $|\mathcal{C}_t|$ is small.

3.4 MonoT5

MonoT5 reframes reranking as a sequence-to-sequence task: the input is `Query:` q `Document:` d `Relevant:` and the model is fine-tuned to generate the token `true` or `false` [Nogueira et al., 2020].

At inference, a softmax is applied only to the logits $z_{\texttt{true}}$ and $z_{\texttt{false}}$ of the first decoding step, and candidates are ranked by the probability of `true`:

$$S_{\text{MonoT5}}(q,d) = \frac{\exp(z_{\texttt{true}})}{\exp(z_{\texttt{true}}) + \exp(z_{\texttt{false}})}.$$

The original work reports that ranking by the raw `true` logit, or using a softmax over the full vocabulary, performs very poorly, so this two-token normalization is part of the method. MonoT5 provides a pointwise reranking baseline from a different model family than MonoBERT.

3.5 DuoBERT (Pairwise Reranking)

DuoBERT reframes reranking as pairwise comparison: the query and two candidates d_i, d_j are fed to BERT as three segments, and the model estimates the probability that d_i is more relevant than d_j [Nogueira et al., 2019]:

$$p_{i,j} = P(d_i \succ d_j \mid q).$$

At inference the pairwise probabilities are aggregated into one score per candidate; we use the SUM rule, which is the main configuration of the original work:

$$S_{\text{DuoBERT}}(q,d_i) = \sum_{j\in J_i} p_{i,j}, \qquad J_i = \{j : j \neq i\}.$$

Pairwise comparison can capture relative distinctions between similar candidates that a pointwise score misses, at the cost of $k_1(k_1-1)$ model calls for k_1 candidates. As in the original multi-stage design, DuoBERT is therefore applied to the top-k_1 candidates of a MonoBERT stage (the original work uses $k_1 = 50$) rather than to all of $\mathcal{C}_t$.

3.6 ColBERTv2 as a Reranker

ColBERTv2 keeps token-level representations and computes late-interaction similarity between the question and a candidate document [Santhanam et al., 2022]. In Section 2 it was described as a first-stage retriever; here the identical scoring function is instead applied only to the FSR candidate set $\mathcal{C}_t$:

$$S_{\text{ColBERT}}(q,d) = \sum_{i\in q} \max_{j\in d} sim(\mathbf{q}_i, \mathbf{d}_j), \qquad d \in \mathcal{C}_t.$$

This provides a direct comparison between a multi-vector late-interaction reranker and GAR's single-score fusion of relevance, coverage, and novelty.

3.7 LambdaMART

LambdaMART is a classical gradient-boosted-tree learning-to-rank algorithm, trained on hand-engineered features (term overlap, document length, prior first-stage retrieval score, and similar signals) rather than learned text representations [Burges, 2010].

$$S_{\text{LambdaMART}}(q, d) = \sum_{\text{trees}} w_{\text{tree}}(\mathbf{x}(q, d)), \qquad d \in \mathcal{C}_t,$$

where $\mathbf{x}(q, d)$ is a hand-engineered feature vector rather than a learned embedding. LambdaMART is included as a non-neural learning-to-rank baseline, since it remains a strong, cheap production reranker in practice and does not depend on the Sentence Transformer model GAR and the other neural baselines share.

3.8 Maximal Marginal Relevance (MMR)

MMR reranks candidates by balancing relevance to the query against redundancy with documents already selected into the output list [Carbonell and Goldstein, 1998]. It is the closest existing baseline to GAR's own relevance-versus-novelty design.

At each step MMR selects the unselected candidate with the largest score,

$$d^\star = \arg\max_{d \in \mathcal{C}_t \setminus S} S_{\text{MMR}}(d),$$

where the score is:

$$S_{\text{MMR}}(d) = \lambda_{\text{MMR}}\, sim(d, q) - (1 - \lambda_{\text{MMR}}) \max_{d_j \in S} sim(d, d_j), \qquad d \in \mathcal{C}_t \setminus S,$$

where S is the set of documents already selected, sim is a similarity function (the original definition allows different similarity functions for the query term and the redundancy term), and $\lambda_{\text{MMR}} \in [0, 1]$ is MMR's own relevance-vs-redundancy trade-off parameter, unrelated to GAR's own scoring weights α, β, γ (Section 4), which play no role in this baseline. For the first selected document, when $S = \varnothing$, the redundancy term $\max_{d_j \in S} sim(d, d_j)$ is defined as 0, so the first document is selected using query relevance alone. MMR has no explicit gap-coverage term: it balances only relevance and redundancy, whereas GAR separately scores relevance, gap coverage, and evidence novelty.

3.9 AIR (Coverage-Driven Iterative Evidence Retrieval)

AIR reformulates its query after each retrieval step by focusing on keywords not yet covered by the evidence already retrieved, so that later steps target complementary evidence rather than repeating earlier ones [Yadav et al., 2020]. AIR directly motivated GAR's coverage term C (Section 4).

In the original method, a candidate P is scored against the current query Q by an IDF-weighted soft alignment over word embeddings:

$$s(Q, P) = \sum_{i=1}^{|Q|} \text{idf}(q_i) \cdot \text{align}(q_i, P), \qquad \text{align}(q_i, P) = \max_{k=1}^{|P|} \cos(q_i, p_k).$$

Query reformulation is driven by the remainder terms, i.e. the query terms not yet covered by the retrieved justifications E_t:

$$Q_r = \tau(Q) \setminus \bigcup_{e \in E_t} \tau(e),$$

where a query term counts as covered if its cosine similarity with some term of e is at least a threshold M (soft matching). The next query is Q_r if $|Q_r| > T$; otherwise Q_r is expanded with the terms of the most recently retrieved justification that are not already in $\tau(Q)$. Retrieval stops when no new query terms are covered or when all query terms are covered. The original work uses $M = 0.95$ and $T = 2$ or $T = 4$ depending on the dataset.

Adapted here as a reranker, AIR is applied to the FSR candidate set $\mathcal{C}_t$: the query Q is the question q, the reformulated query Q_r (with the expansion rule above) is built from the current evidence E_t, and each $d \in \mathcal{C}_t$ is ranked by $s(Q_r, d)$. Dividing by $|Q_r|$ would not change the ranking and is omitted. If $Q_r = \varnothing$ (all query terms are covered), AIR's stopping criterion is met and no reformulated query exists; in that case we keep the order produced by first-stage retrieval. AIR does not use the gap G_t: its own notion of missing information is Q_r. Unlike GAR, AIR has no separate relevance, coverage, and novelty terms; coverage enters only through query reformulation.

3.10 Dartboard / RIG (Relevant Information Gain)

Dartboard selects a set of passages that maximizes the total relevant information gained, so that redundancy is penalized implicitly rather than through an explicit trade-off weight [Pickett et al., 2024]. It is modeled as a game: a target passage T is drawn with probability $P(T = t \mid q, \sigma) = \mathcal{N}(q, t, \sigma)$, the query lands near it, and a set of guesses G is scored by how close its closest guess is to the target. With a Gaussian kernel the objective (paper Eq. 2) is, up to sign and constants:

$$s(G, q, A, \sigma) \;\propto\; \sum_{t \in A} \mathcal{N}(q, t, \sigma) \max_{g \in G} \mathcal{N}(t, g, \sigma),$$

which is maximized, where A is the set of points considered (here the candidate set $\mathcal{C}_t$), σ is the standard deviation of the kernel, and $\mathcal{N}$ is computed from the distance between embeddings (cosine-based in the original).

The method is applied greedily. The first selected passage is the one closest to the query:

$$d_1 = \arg\max_{d \in \mathcal{C}_t} \mathcal{N}(q, d, \sigma).$$

Each subsequent passage maximizes the score of the enlarged set:

$$S_{\text{Dartboard}}(d \mid G) = \sum_{t \in \mathcal{C}_t} \mathcal{N}(q, t, \sigma) \max\Big(\max_{g \in G} \mathcal{N}(t, g, \sigma),\ \mathcal{N}(t, d, \sigma) \Big), \qquad d \in \mathcal{C}_t \setminus G,$$

and the reranked list is the order of selection. The original implementation works in log space with a log-sum-exp for numerical stability, which gives the same ranking. Because the maximum ensures that an exact duplicate adds no information, duplicates are not selected until all non-duplicates are exhausted. As in the original, Dartboard uses neither the collected evidence E_t nor the gap G_t; it is included because it directly motivated GAR's novelty term N (Section 4). Its only hyperparameter is σ, and the original work shows that it generalizes both nearest-neighbor retrieval and MMR.

3.11 RankT5

RankT5 fine-tunes T5 so that it outputs a real-valued ranking score for each query–document pair instead of generating a text token [Zhuang et al., 2023]. Two structures are studied: an encoder–decoder model that uses the first output token of the decoder, and an encoder-only model that omits the decoder.

$$S_{\text{RankT5}}(q, d) = f_\theta(q, d), \qquad d \in \mathcal{C}_t,$$

where f_θ is fine-tuned with a ranking objective: pointwise cross-entropy, pairwise logistic, or listwise softmax cross-entropy. The pairwise and listwise losses use groups of candidates only during training; at inference each pair is scored independently. RankT5 represents the learning-to-rank family of rerankers.

3.12 RankGPT (LLM Listwise Reranking)

RankGPT prompts a large language model with the question and a group of passages, each labelled with an identifier ([1], [2], ...), and asks it to output the identifiers in descending order of relevance, for example [2] > [3] > [1], without producing intermediate relevance scores [Sun et al., 2023].

$$(p_1, \ldots, p_w) \xrightarrow{\text{LLM permutation generation}} [\pi_1] > [\pi_2] > \cdots > [\pi_w].$$

Because of context-length limits, a candidate list of M passages is reranked with a sliding window of size w and step size s, from back to front: the passages ranked $M - w$ to M are ranked first, then the window moves toward the top by s positions, so the best passages of one window carry into the next. The original work uses $w = 20$, $s = 10$ on the top-100 first-stage passages, and reports that the result depends strongly on the initial order (the first-stage order). If the model omits identifiers, the missing passages are appended in their original order. RankGPT is included because MIGRES is itself LLM-driven, making an LLM-based reranking baseline a natural comparison point.

3.13 Reranking Baseline Comparison

System	Reranking type	Role
MIGRES+FSR	No reranking	Control
MIGRES+FSR+MonoBERT	Cross-encoder pointwise	Reranking baseline
MIGRES+FSR+MonoT5	Seq2seq pointwise	Reranking baseline
MIGRES+FSR+DuoBERT	Pairwise	Reranking baseline
MIGRES+FSR+ColBERTv2	Late interaction	Reranking baseline
MIGRES+FSR+LambdaMART	Classical learning-to-rank	Reranking baseline
MIGRES+FSR+RankT5	Neural learning-to-rank	Reranking baseline
MIGRES+FSR+MMR	Relevance vs. redundancy	Closest conceptual baseline
MIGRES+FSR+AIR	Gap-coverage-driven	Direct precursor to C
MIGRES+FSR+Dartboard/RIG	Relevance vs. novelty	Direct precursor to N
MIGRES+FSR+RankGPT	LLM listwise	Reranking baseline
MIGRES+FSR+GAR	Gap-aware reranking	Proposed method

All rerankers are applied to the identical FSR candidate set $\mathcal{C}_t$ for each question, so that only the reranking method varies across systems. AIR and Dartboard/RIG are included not merely as baselines but as the two prior works that directly motivated GAR's coverage and novelty terms; the comparison therefore also tests whether GAR's fused three-signal score outperforms each of its own inspirations used alone.

4 Development and Introduction of GAR

4.1 What GAR Is

GAR is a reranking algorithm designed for iterative retrieval-augmented generation. It reranks the candidate set $\mathcal{C}_t$ returned by FSR's first-stage retrieval, rather than searching the corpus $\mathcal{D}$ directly. It changes the ranking question from:

"Is this document relevant to the original question?"

to:

"Is this document useful given what is missing and what is already known?"

GAR uses three signals:

1. **Relevance**: relevance to the original question.
2. **Gap coverage**: how much of the missing information is covered by the candidate.
3. **Evidence novelty**: how much new information the candidate adds beyond existing evidence.

4.2 Why GAR Was Developed

A query-only retriever can repeatedly return documents that are strongly related to the question but do not resolve the current missing information. MIGRES already exposes the missing-information state, making it possible for the retrieval algorithm to use that state directly [Wang et al., 2025].

The motivation for the GAR components comes from three sources.

First, MIGRES provides the missing-information concept: later retrieval can be targeted toward information that is still missing [Wang et al., 2025].

Second, Yadav et al. use an iterative evidence-retrieval process that focuses later retrieval on information not covered by existing justifications. This motivates the coverage component in GAR [Yadav et al., 2020].

Third, Relevant Information Gain argues that retrieval should avoid wasting limited context on redundant information while retaining relevance [Pickett et al., 2024]. This motivates the novelty component.

GAR combines these ideas into one retrieval score. The formulation is therefore a proposed unified model; the project does not claim that each component is independently novel.

4.3 Applications

GAR is intended for settings where information needs change during retrieval, including:

- multi-hop question answering;
- iterative RAG;
- research-oriented evidence retrieval;
- evidence gathering over large document collections;
- systems that explicitly track missing information.

GAR does not require a new language model. The surrounding system can provide the current gap and evidence state.

5 Mathematical Formulation

5.1 Notation

At retrieval iteration t:

$$q = \text{original question},$$

$$E_t = \text{evidence collected before iteration } t,$$

$$G_t = \{g_1, \ldots, g_m\}$$

is the set of current information requirements, and

$$\mathcal{D} = \text{searchable document corpus.}$$

The first-stage retriever performs retrieval over $\mathcal{D}$, using K_0 as the retrieval depth for each individual query (the question or a single gap requirement), and returns a candidate set:

$$\mathcal{C}_t \subseteq \mathcal{D}, \qquad |\mathcal{C}_t| \leq (m+1)K_0.$$

As Eq. (10) makes explicit, $\mathcal{C}_t$ is the deduplicated union of $m+1$ separate top-K_0 retrievals (one for q and one for each $g_i \in G_t$), so its size is bounded above by $(m+1)K_0$ rather than equal to K_0; K_0 should be read as the per-query retrieval depth, not the final candidate-pool size.

A candidate document is a member of this candidate set, not of the full corpus directly:

$$d \in \mathcal{C}_t.$$

An individual evidence item is:

$$e \in E_t.$$

The GAR input is therefore:

$$\boxed{(q, E_t, G_t, \mathcal{C}_t)}.$$

GAR never receives $\mathcal{D}$ directly; $\mathcal{D}$ is used only by the first-stage retriever to construct $\mathcal{C}_t$ (Section 6). Which concrete algorithm performs this first-stage retrieval is itself an empirical choice, resolved by the retriever comparison in Section 2 before the reranking comparison in Section 3 and Section 8 is run; once selected, that retriever is held fixed so that only the reranking method varies across the main comparison.

5.2 Text Embeddings

GAR needs a fixed representation function:

$$\phi : \mathcal{T} \to \mathbb{R}^p.$$

We use:

`sentence-transformers/multi-qa-mpnet-base-cos-v1`

as the fixed embedding model. The model maps sentences and paragraphs to 768-dimensional dense vectors and was designed for semantic search [Reimers and Gurevych, 2019, Sentence Transformers, 2020].

The same model is used for questions, gaps, candidate documents, and evidence. Candidate document embeddings are computed on demand for $\mathcal{C}_t$; GAR does not maintain a corpus-wide embedding index, since the first-stage retriever operates directly on raw text.

5.3 Cosine Similarity

For two text items x and y:

$$\cos(x, y) = \frac{\phi(x)^T \phi(y)}{\|\phi(x)\|_2 \|\phi(y)\|_2}.$$

For a normalized score in $[0, 1]$, GAR uses:

$$\boxed{s(x, y) = \frac{1}{2}\left(1 + \frac{\phi(x)^T \phi(y)}{\|\phi(x)\|_2 \|\phi(y)\|_2}\right)} \tag{1}$$

The embedding model creates vectors; cosine similarity compares them. This formula assumes both $\phi(x)$ and $\phi(y)$ are nonzero, which holds in practice for the embedding model used here.

5.4 Relevance

Relevance measures how well a candidate answers the original question, independent of what is already known or still missing:

$$\boxed{R(d, q) = s(d, q)} \tag{2}$$

Relevance is deliberately simple: it reuses the single similarity primitive from Eq. (1) with no additional blending or hyperparameter. Whether a candidate also addresses the current information gap is a separate question, answered by the Coverage term below — keeping the two concerns apart avoids mixing two different notions of "relevant" into one number.

5.5 Gap Coverage

Let $\tau(x)$ be the set of normalized content tokens obtained after the fixed preprocessing pipeline.

For a non-empty gap requirement:

$$\boxed{c(d, g_i) = \frac{|\tau(d) \cap \tau(g_i)|}{|\tau(g_i)|}} \tag{3}$$

This is normalized token recall. The coverage score is the average recall across all gap requirements:

$$\boxed{C(d, G_t) = \frac{1}{m} \sum_{i=1}^{m} c(d, g_i)} \tag{4}$$

Thus $C(d, G_t) \in [0, 1]$.

If $\tau(g_i)$ is empty, that invalid requirement is removed before retrieval. Eq. (4) requires $m > 0$; if $G_t = \varnothing$ after removing invalid requirements, MIGRES has determined that no information is currently missing, and the iterative retrieval loop stops for that question rather than evaluating C on an empty gap set.

5.6 Evidence Novelty

For non-empty evidence:

$$\boxed{N(d, E_t) = 1 - \max_{e \in E_t} s(d, e)} \tag{5}$$

A candidate very similar to existing evidence receives low novelty. A candidate dissimilar to all existing evidence receives high novelty.

At the first iteration:

$$E_0 = \emptyset.$$

We define:

$$\boxed{N(d, \emptyset) = 1.} \tag{6}$$

5.7 Final GAR Score

The final score is:

$$\boxed{S(d \mid q, E_t, G_t) = \alpha R(d, q) + \beta C(d, G_t) + \gamma N(d, E_t)} \tag{7}$$

subject to:

$$\boxed{\alpha, \beta, \gamma \geq 0, \qquad \alpha + \beta + \gamma = 1.} \tag{8}$$

Because the three components are normalized, the final score is also in $[0, 1]$.

5.8 Hyperparameters

GAR has three scoring hyperparameters:

$$\boxed{\alpha, \beta, \gamma}$$

and one retrieval hyperparameter:

$$\boxed{K_0}.$$

Their roles are:

Parameter	Role	Constraint
α	Weight of relevance R	≥ 0
β	Weight of coverage C	≥ 0
γ	Weight of novelty N	≥ 0
K_0	First-stage retrieval depth per query (question or gap requirement); candidate pool $\lvert\mathcal{C}_t\rvert \leq (m+1)K_0$	$K_0 \geq k$

The final retrieval depth k is an **experimental setting**, not a GAR-specific scoring parameter. It should be kept the same across systems for fair comparison.

The fixed embedding model and cosine similarity are also configuration choices, not learned GAR hyperparameters.

6 How GAR Works Using MIGRES

6.1 Overall Architecture

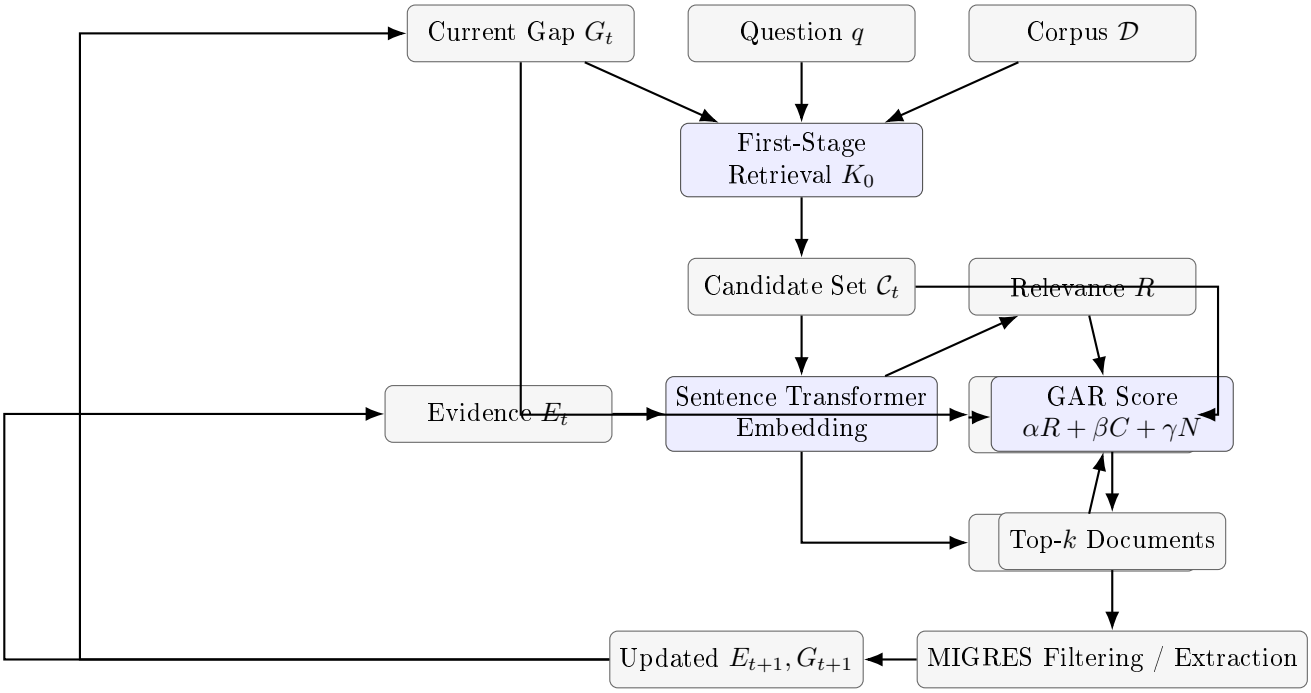


Figure 1: Overall architecture of GAR within the MIGRES framework. MIGRES provides the current question, evidence, and information gap. The selected first-stage retriever (Section 2) retrieves over the corpus $\mathcal{D}$ to produce the candidate set $\mathcal{C}_t$. Relevance and Novelty are computed from Sentence Transformer embeddings of the candidates; Coverage is computed directly from candidate and gap text via token overlap and does not require embeddings. The reranked documents are then returned to MIGRES for filtering, extraction, and state updating.

6.2 Step 1: Dataset Question

A benchmark supplies the original question:

$$q.$$

The benchmark's gold answer and supporting evidence are not given to GAR during retrieval.

6.3 Step 2: MIGRES Creates the Retrieval State

MIGRES processes the question and maintains:

$$E_t = \text{evidence already collected}$$

and:

$$G_t = \text{information still missing.}$$

The gap can contain multiple requirements:

$$G_t = \{g_1, \ldots, g_m\}.$$

GAR does not independently invent G_t in the main experiment. It receives the gap produced by MIGRES.

6.4 Step 3: First-Stage Retrieval

The selected first-stage retriever (Section 2) operates directly on the raw text of $\mathcal{D}$; for the sparse lexical candidates considered here (e.g. BM25), it requires no embeddings. For each query-like input x, it returns the K_0 highest-scoring documents:

$$\mathrm{FSR}_{K_0}(x, \mathcal{D}).$$

GAR's candidate set is built using both the original question and every gap requirement, so that first-stage retrieval is itself gap-aware:

$$\boxed{\mathcal{C}_t = \mathrm{FSR}_{K_0}(q, \mathcal{D}) \cup \bigcup_{i=1}^{m} \mathrm{FSR}_{K_0}(g_i, \mathcal{D})} \tag{9}$$

Duplicates are removed. Because each of the $m+1$ retrieval calls can return up to K_0 documents, the resulting candidate pool satisfies $|\mathcal{C}_t| \leq (m+1)K_0$ after deduplication, not $|\mathcal{C}_t| = K_0$; K_0 is the retrieval depth used for each individual query, while k is the final number of documents returned after reranking.

Taking the union in Eq. (9) discards the individual first-stage scores each document received under q or under a given g_i, since a document can appear in more than one of the $m+1$ retrieved lists with different scores. GAR itself does not need these first-stage scores, as it rescoring every $d \in \mathcal{C}_t$ from scratch using R, C, and N. However, the no-reranking control (the selected first-stage retriever with no reranker) does need a defined top-k over $\mathcal{C}_t$: for that system, we fuse a document's score as the maximum first-stage score it received across the retrieval calls that returned it, i.e. $\max\big(\mathrm{FSR}(d, q), \max_i \mathrm{FSR}(d, g_i)\big)$, and rank $\mathcal{C}_t$ by this fused score.

This step is cheap relative to embedding the full corpus even when $\mathcal{D}$ contains millions of documents, and it avoids requiring GAR to build or query a dense index over the full corpus.

6.5 Step 4: Convert Candidate Text to Embeddings

GAR uses the fixed Sentence Transformer model:

$$\phi(x).$$

The question, each gap requirement, each evidence item, and *only the documents already in* $\mathcal{C}_t$ are represented as vectors. Because $|\mathcal{C}_t| \leq (m+1)K_0 \ll |\mathcal{D}|$, GAR never needs to embed or index the full corpus — embeddings are computed on demand for the small candidate set returned by the first-stage retriever.

6.6 Step 5: Calculate Relevance

For every candidate:

$$R(d, q)$$

measures:

question-document relevance.

6.7 Step 6: Calculate Coverage

GAR then asks:

"Does this document cover the information that is missing?"

This produces:

$$C(d, G_t).$$

6.8 Step 7: Calculate Novelty

GAR then asks:

"Is this document redundant with evidence already collected?"

This produces:

$$N(d, E_t).$$

6.9 Step 8: Calculate the GAR Score

For every candidate:

$$S(d \mid q, E_t, G_t) = \alpha R + \beta C + \gamma N.$$

The candidate with the highest score is preferred because it is simultaneously relevant, gap-covering, and non-redundant.

6.10 Step 9: Return Top-k

GAR returns:

$$\boxed{T_t = \mathrm{TopK}_{d \in \mathcal{C}_t} S(d \mid q, E_t, G_t)} \tag{10}$$

where k is fixed for the experiment.

6.11 Step 10: MIGRES Continues

MIGRES receives the retrieved documents and performs its normal filtering and extraction.
It updates:

$$E_{t+1}$$

and, if more information is needed:

$$G_{t+1}.$$

GAR is then called again.

The complete loop is:

$$\boxed{(q, E_t, G_t, \mathcal{D}) \xrightarrow{\text{FSR}} \mathcal{C}_t \xrightarrow{\text{GAR}} T_t \xrightarrow{\text{MIGRES}} (E_{t+1}, G_{t+1}) \xrightarrow{\text{FSR}} \mathcal{C}_{t+1} \xrightarrow{\text{GAR}} T_{t+1}}$$

This is why GAR is evaluated inside MIGRES rather than as an isolated one-shot reranker.

6.12 Retrieval vs. Reranking in This Design

The two stages must not be confused:

$$\boxed{\text{First-stage retrieval} \rightarrow \text{Embedding} \rightarrow \text{GAR scoring} \rightarrow \text{Top-}k}$$

The first-stage retriever finds a manageable candidate set from the full corpus $\mathcal{D}$.
Embedding converts only that candidate set (plus q, G_t, E_t) into vectors.
GAR makes the final ranking decision over the candidate set.
This is precisely the reranking design proposed in this project: a first-stage retriever narrows the corpus down to $\mathcal{C}_t$, and GAR reranks that output using relevance, gap coverage, and evidence novelty.
A version of GAR that performs its own first-stage candidate generation directly from $\mathcal{D}$ (for example, via an ANN index over GAR's own embeddings, without relying on a separate first-stage retriever) would instead be a complete end-to-end retrieval algorithm; that variant is not evaluated here and is left as a direction for future work.

6.13 Development, Validation, and Test Protocol

6.13.1 Is GAR Trained?

No.
The proposed GAR algorithm has no trainable parameters. We do not train GAR on D_{train}.
The embedding model is a fixed pretrained Sentence Transformer. It is not fine-tuned in the primary experiment.
Therefore the experiment has no GAR training stage.

6.13.2 Training Data

If a baseline uses an official pretrained checkpoint, we use that checkpoint rather than retraining it on the evaluation data.
The benchmark training split may be used for auxiliary development or baseline-specific training only if required by the official implementation. Any such use must be documented and must not use test annotations.
For GAR itself:

$$D_{\text{train}} \not\rightarrow \text{GAR parameter learning.}$$

6.13.3 Validation Set D_{val}

We define:

$$\boxed{D_{\text{val}} = \text{the held-out development/validation split.}}$$

Where an official development split exists, we use it. The exact files and split names are recorded for each dataset.
GAR hyperparameters are selected using the *complete MIGRES+FSR+GAR pipeline* on D_{val}.
This is important because GAR cannot be meaningfully validated in isolation: E_t and G_t are produced by the MIGRES iterative process, and $\mathcal{C}_t$ is produced by FSR.
The validation loop is:

$$D_{\text{val}} \rightarrow \text{MIGRES} \rightarrow (q, E_t, G_t) \rightarrow \text{FSR} \rightarrow \mathcal{C}_t \rightarrow \text{GAR} \rightarrow \text{MIGRES} \rightarrow \text{validation metric.}$$

6.13.4 Bayesian Optimization

We use Bayesian optimization to select:

$$\theta_{\text{GAR}} = \{\alpha, \beta, \gamma, K_0\}.$$

Bayesian optimization uses previous validation evaluations to decide which configuration to test next rather than exhaustively evaluating every possible combination [Snoek et al., 2012].
The search space is:

$$\alpha, \beta, \gamma \geq 0, \qquad \alpha + \beta + \gamma = 1,$$

and a predefined discrete candidate set for K_0, for example:

$$K_0 \in \{50, 100, 200, 500\}.$$

The exact search space and random seed are recorded before the experiment.
The optimization objective must be selected before test evaluation. The primary objective should be a retrieval metric aligned with the main research question, such as Recall@k on D_{val}.
The procedure is:

$$\boxed{D_{\text{val}} \rightarrow \text{MIGRES+FSR+GAR} \rightarrow \text{validation score} \rightarrow \text{Bayesian optimization} \rightarrow \theta^*_{\text{GAR}}}$$

The selected configuration is then frozen.

6.13.5 Test Set D_{test}

After validation:

$$\boxed{\theta^*_{\text{GAR}} \text{ is frozen.}}$$

We then evaluate:

$$D_{\text{test}} \rightarrow \text{MIGRES+FSR+GAR}(\theta^*_{\text{GAR}}) \rightarrow \text{final metrics.}$$

No test result is used to change the hyperparameters.
The same k, dataset, corpus, MIGRES configuration, and stopping conditions are used for the comparison systems wherever possible.
Because K_0 is one of the parameters selected by Bayesian optimization on D_{val} for GAR specifically (Section 6.13.2), and the same frozen K_0 then fixes the candidate pool $\mathcal{C}_t$ passed to every baseline reranker, the candidate-pool depth is not independently tuned for each baseline. This is a limitation of the comparison: it should be read as GAR vs. baselines under a candidate-pool depth chosen to favor GAR's own validation performance, not under a depth optimized separately for each system. A stricter design would either fix K_0 in advance independently of any reranker's performance, or tune each baseline's own hyperparameters (including its effective candidate depth, where applicable) on D_{val} under the same protocol used for GAR.

6.13.6 Complete Experimental Flow

1. Prepare the official dataset splits and searchable corpus.
2. Run the first-stage-retriever comparison (Section 2) on D_{val} and select FSR as the best-performing candidate; freeze FSR for every step below.
3. Reproduce MIGRES+FSR as the no-reranking control.
4. Verify the MIGRES retrieval interface and confirm how E_t and G_t are exposed.
5. Implement on-demand embedding of FSR's candidate set $\mathcal{C}_t$ using the fixed Sentence Transformer.

6. Implement and unit-test the GAR equations.
7. Run Bayesian optimization on D_{val} through the complete MIGRES+FSR+GAR pipeline.
8. Freeze $\alpha, \beta, \gamma, K_0$.
9. Run the held-out D_{test} with the frozen configuration.
10. Run MonoBERT, MonoT5, DuoBERT, ColBERTv2, LambdaMART, RankT5, MMR, AIR, Dartboard/RIG, and RankGPT as rerankers on the identical FSR candidate set, using their established implementations.
11. Evaluate retrieval quality, downstream answer quality, efficiency, and cost.
12. Run GAR ablations using the same frozen evaluation protocol.

7 Datasets

7.1 Dataset Selection

The project uses the six datasets reported in the MIGRES evaluation:

WikiHop, HotpotQA, MuSiQue, Natural Questions, TriviaQA, StrategyQA

These datasets provide questions and task-specific answer/evidence annotations suitable for evaluating retrieval and downstream question answering.

7.2 WikiHop

Paper: Welbl et al. [2018].
WikiHop is a multi-document, multi-hop reading-comprehension benchmark designed so that the answer requires combining information across documents. The dataset is useful for GAR because it tests whether retrieval can assemble complementary evidence rather than repeatedly retrieving one topic.
Typical fields include a question, candidate documents, an answer, and supporting information.
Paper: https://aclanthology.org/Q18-1021/

7.3 HotpotQA

Paper: Yang et al. [2018].
HotpotQA contains about 113K Wikipedia-based question-answer pairs and provides sentence-level supporting facts. It includes multi-hop and comparison questions [Yang et al., 2018].
For first-stage development, we use the FullWiki setting because it requires retrieval from a larger Wikipedia corpus. The official repository provides train, development, and FullWiki resources and explicitly allows researchers to replace its retrieval component with their own IR system.
Paper: https://aclanthology.org/D18-1259/
Official resources: https://hotpotqa.github.io/
The repository documentation describes the FullWiki development and test files and the use of custom IR systems.

7.4 MuSiQue

Paper: Trivedi et al. [2022].
MuSiQue was designed to reduce shortcut solutions in multi-hop QA. MuSiQue-Ans contains about 25K questions requiring 2–4 hops [Trivedi et al., 2022].
Typical information includes a question, answer, reasoning structure, and supporting evidence.
Paper: https://aclanthology.org/2022.tacl-1.31/
Official repository: https://github.com/StonyBrookNLP/musique

7.5 Natural Questions

Paper: Kwiatkowski et al. [2019].
Natural Questions consists of real anonymized queries issued to Google Search and Wikipedia-based annotations. The public release contains 307,373 training examples, 7,830 development examples, and 7,842 sequestered test examples [Kwiatkowski et al., 2019].
Typical fields include a question, Wikipedia page information, long-answer annotations, short-answer annotations, and document-level information.
Paper: https://aclanthology.org/Q19-1026/
Official research page: https://ai.google/research/NaturalQuestions

7.6 TriviaQA

Paper: Joshi et al. [2017].
TriviaQA contains about 95K question-answer pairs and more than 650K question-answer-evidence triples. Questions were written by trivia enthusiasts and paired with independently gathered evidence documents [Joshi et al., 2017].
Typical fields include a question, answer, aliases, and evidence documents.
Paper: https://aclanthology.org/P17-1147/
Official resources: https://nlp.cs.washington.edu/triviaqa/

7.7 StrategyQA

Paper: Geva et al. [2021].
StrategyQA contains 2,780 questions with implicit reasoning strategies. Each example includes a question, a reasoning decomposition, and Wikipedia evidence paragraphs [Geva et al., 2021].
This makes it useful for testing whether gap-aware reranking helps when the required reasoning steps are not directly stated in the question.
Paper: https://aclanthology.org/2021.tacl-1.21/
Official resources: https://allenai.org/data/strategyqa

7.8 Dataset Summary

Dataset	Scale	Useful properties for GAR
WikiHop	∼51K	Multi-document, multi-hop reasoning.
HotpotQA	∼113K	Multi-hop QA with sentence-level supporting facts; FullWiki retrieval.
MuSiQue	∼25K	2–4 hop compositional reasoning.
Natural Questions	307,373 train; 7,830 dev; 7,842 test	Real search questions with Wikipedia annotations.
TriviaQA	∼95K QA pairs; >650K evidence triples	Heterogeneous evidence and lexical variation.
StrategyQA	2,780	Implicit reasoning with evidence paragraphs.

Exact counts used in experiments must be taken from the downloaded release and recorded in the experiment configuration. Dataset counts can differ by release or preprocessing.

7.9 What a Dataset Example Looks Like

The common conceptual structure is:

Question + Corpus/Evidence + Answer + Ground-Truth Evidence/Annotations

For example, a HotpotQA-style example contains a natural-language question, an answer, supporting facts, and Wikipedia context. During reranking, GAR receives the question, MIGRES's evidence

and gap state, and the candidate set already retrieved by FSR from the searchable corpus. The gold answer and gold supporting evidence are reserved for evaluation.
The ground-truth evidence must never be used to construct G_t, E_t, or the FSR candidate pool during test-time retrieval.

7.10 Dataset Split Policy

For each dataset:

$$D_{\text{train}}, \quad D_{\text{val}}, \quad D_{\text{test}}$$

are treated separately where official splits are available.
For GAR:

$$D_{\text{val}}$$

is used for hyperparameter optimization, while:

$$D_{\text{test}}$$

is used only for the final evaluation.
If an official public test set has hidden answers, its evaluation follows the dataset's official evaluation procedure.

8 Experiments

8.1 Research Questions

RQ1. Does reranking FSR's candidates with GAR improve retrieval quality over the MIGRES FSR baseline with no reranking?

$$\boxed{\text{MIGRES+FSR+GAR} > \text{MIGRES+FSR?}}$$

RQ2. Does GAR outperform established reranking methods applied to the same FSR candidates?

$$\text{GAR} \quad \text{vs.} \quad \left\{ \begin{array}{c} \text{MonoBERT, MonoT5, DuoBERT, ColBERTv2, LambdaMART,} \\ \text{RankT5, MMR, AIR, Dartboard/RIG, RankGPT} \end{array} \right\}.$$

RQ3. Does reranking FSR's output with GAR improve downstream MIGRES answer quality, compared with no reranking?
RQ4. How does GAR's performance change when relevance, coverage, or novelty is removed?
RQ5. Does GAR generalize across the six datasets?
RQ6. What is the computational cost of GAR compared with the baselines?
RQ7. Does GAR improve retrieval without adding unnecessary LLM calls?

8.2 Hypotheses

Primary hypothesis:

$$H_0: \quad \text{GAR reranking does not improve performance over FSR with no reranking.}$$

$$H_1: \quad \text{GAR reranking improves performance over FSR with no reranking.}$$

Secondary hypothesis:

$$H_0: \quad \text{GAR} \leq \text{best reranking baseline.}$$

$$H_1: \quad \text{GAR} > \text{best reranking baseline.}$$

Here "best reranking baseline" is determined on D_{val}, before D_{test} is inspected: the ten established rerankers of Section 3 are ranked by their own validation-set performance on the primary retrieval metric (Section 6.13.4), and the validation-set winner is frozen as the single comparator used in this hypothesis test on D_{test}. Selecting the comparator by its test-set performance instead would make this a post-selection comparison and invalidate the significance level.
For ablations:

$$H_0: \quad \text{removing a GAR component does not change performance.}$$

$$H_1: \quad \text{removing a GAR component reduces performance.}$$

We report effect sizes and confidence intervals together with statistical significance. All comparisons use a paired bootstrap test on the per-question primary retrieval metric, with 10,000 resamples; this test is fixed before inspecting D_{test} results. Only the primary hypothesis (GAR vs. no reranking) is treated as confirmatory, tested one-sided at $\alpha_{\text{sig}} = 0.05$ to match its directional formulation (H_1 : GAR > baseline). The secondary (best-baseline) hypothesis and the individual ablations are treated as secondary/exploratory comparisons: we report their bootstrap p-values and confidence intervals for interpretation, but apply a Holm step-down correction across that family before declaring any of them significant at the family-wise $\alpha_{\text{sig}} = 0.05$ level.

8.3 Main Comparison

The main experiment compares:

MIGRES+FSR (no rerank)
MIGRES+FSR+MonoBERT
MIGRES+FSR+MonoT5
MIGRES+FSR+DuoBERT
MIGRES+FSR+ColBERTv2
MIGRES+FSR+LambdaMART
MIGRES+FSR+RankT5
MIGRES+FSR+MMR
MIGRES+FSR+AIR
MIGRES+FSR+Dartboard/RIG
MIGRES+FSR+RankGPT
MIGRES+FSR+GAR

The same questions, corpus, k, K_0, MIGRES settings, answer-generation model, and stopping conditions should be used wherever possible, so that only the reranking method varies across systems.

8.4 Ablation Studies

The full GAR model is:

$$S_{\text{GAR}} = \alpha R + \beta C + \gamma N.$$

We remove one component at a time.
Without relevance:

$$\boxed{S_{-R}(d) = \beta C(d, G_t) + \gamma N(d, E_t)}$$

Without coverage:

$$\boxed{S_{-C}(d) = \alpha R(d,q) + \gamma N(d, E_t)}$$

Without novelty:

$$\boxed{S_{-N}(d) = \alpha R(d,q) + \beta C(d, G_t)}$$

In each ablation, α, β, and γ are the same values already frozen from the full GAR model (Section 6.13.5); we do not renormalize the remaining weights to sum to one before removing a component. Renormalizing by dividing by the sum of the retained weights would multiply every candidate's score by the same positive constant whenever that sum is positive, and therefore would not change the ranking; we keep the original frozen weights for this reason. An ablation is degenerate only when the retained weights sum to zero (e.g. $\beta + \gamma = 0$ for S_{-R}), in which case every candidate receives a score of zero.
We also recommend one additional diagnostic ablation:

$$GAR_{\text{R-only}}$$

which ranks by question relevance $R(d,q)$ alone (equivalent to $\alpha = 1$, $\beta = \gamma = 0$). Together with S_{-N} (relevance plus coverage), it helps determine whether the main benefit comes from adding gap coverage or from the complete relevance–coverage–novelty combination.

8.5 Candidate-Pool Sensitivity

Because K_0 controls how many candidates reach the full GAR scorer, we test:

$$K_0 \in \{50, 100, 200, 500\}$$

on the validation set during optimization.
The test experiment uses only the selected value.
We report both retrieval quality and latency as K_0 changes.

8.6 Retrieval Metrics

8.6.1 Recall@k

Recall@k measures how much of the ground-truth relevant evidence appears in the first k retrieved documents:

$$\boxed{Recall@k = \frac{|\text{relevant evidence} \cap \text{top-}k|}{|\text{relevant evidence}|}}$$

This is a primary metric because GAR is designed to retrieve missing evidence. Benchmarks used here provide at least one relevant-evidence item per question, so this denominator is not expected to be zero in practice; if an instance with no annotated relevant evidence does occur, that instance is excluded from the Recall@k average rather than reported as 0/0.

8.6.2 Precision@k

$$\boxed{Precision@k = \frac{|\text{relevant documents in top-}k|}{k}}$$

Precision measures how much of the retrieved set is relevant.

8.6.3 MRR

Mean Reciprocal Rank measures how early the first relevant result appears:

$$\boxed{MRR = \frac{1}{N}\sum_{i=1}^{N}\frac{1}{rank_i}}$$

where $rank_i$ is the rank of the first relevant result for question i. If no relevant document appears among the retrieved results for question i, we define $\frac{1}{rank_i} = 0$ for that question, following standard practice.

8.6.4 nDCG@k

For graded relevance:

$$DCG@k = \sum_{i=1}^{k}\frac{2^{rel_i}-1}{\log_2(i+1)}$$

and:

$$\boxed{nDCG@k = \frac{DCG@k}{IDCG@k}}$$

where $IDCG@k$ is the ideal DCG at cutoff k. The cumulated-gain idea is due to Järvelin and Kekäläinen [2002]; the exponential-gain form $2^{rel_i}-1$ with a $\log_2(i+1)$ discount used here is the variant common in learning to rank [Burges et al., 2005], and differs from the linear-gain, $\log_b$-discount definition of the original paper. If $IDCG@k = 0$ (no relevant document exists for that question), we define $nDCG@k = 0$ for that question rather than leaving the ratio undefined.

8.6.5 Evidence Coverage

Because GAR explicitly models an information gap, we additionally report an evidence-coverage measure where the dataset annotations permit it:

$$\boxed{GapEvidenceCoverage@k = \frac{\text{ground-truth evidence requirements retrieved in top-}k}{\text{total ground-truth evidence requirements}}}$$

The exact mapping from benchmark annotations to evidence requirements must be defined separately for each dataset. As with Recall@k, an instance with zero total ground-truth evidence requirements is excluded from the $GapEvidenceCoverage@k$ average rather than reported as 0/0.

8.7 Downstream QA Metrics

The final MIGRES answer is evaluated using the metrics supported by each benchmark.
These may include:

- Exact Match (EM);
- token-level F1;
- answer accuracy for classification-style datasets;
- evidence/supporting-fact quality where annotations permit.

For a benchmark with a normalized answer set A_i:

$$EM_i = \mathbf{1}[\hat{a}_i \in A_i].$$

Average exact match is:

$$EM = \frac{1}{N}\sum_{i=1}^{N} EM_i.$$

For token-level F1, precision and recall are computed over normalized answer tokens and then combined as:

$$F1 = \frac{2PR}{P+R}.$$

$F1$ is undefined when $P+R=0$ (e.g. both predicted and gold answers are empty after normalization); we adopt whatever convention the benchmark's official evaluation script uses for this case (typically $F1 = 1$ if both are empty and $F1 = 0$ if only one is), and that script should be used whenever available.

8.8 Efficiency Metrics

We report:

- retrieval latency per question;
- first-stage retriever indexing time;
- index memory;
- number of retrieval iterations;
- number of documents scored by GAR;
- number of LLM calls in the complete MIGRES system;
- input and output tokens;
- API cost where applicable.

The GAR scorer itself should not require one LLM call per candidate. Embeddings, cosine similarity, lexical coverage, and novelty can be calculated locally.

8.9 API Cost

MIGRES may issue multiple LLM calls per question (e.g. for gap identification, extraction, and solving), and these calls need not share the same model or price. Let $\mathcal{L}(q)$ be the set of LLM calls MIGRES makes while answering question q, and for call $\ell \in \mathcal{L}(q)$ let $T_{\text{in}}(\ell)$, $T_{\text{out}}(\ell)$ be its input/output token counts and $p_{\text{in}}(\ell)$, $p_{\text{out}}(\ell)$ the corresponding per-token prices of whichever model served that call. The total cost for question q is:

$$\boxed{Cost(q) = \sum_{\ell \in \mathcal{L}(q)} \Big(T_{\text{in}}(\ell)p_{\text{in}}(\ell) + T_{\text{out}}(\ell)p_{\text{out}}(\ell) \Big)}$$

This reduces to the single-call, single-model form $Cost(q) = T_{\text{in}}(q)p_{\text{in}} + T_{\text{out}}(q)p_{\text{out}}$ only when $|\mathcal{L}(q)| = 1$.

Average cost is:

$$\boxed{\overline{Cost} = \frac{1}{N}\sum_{i=1}^{N} Cost(q_i)}$$

The comparison must distinguish GAR's local retrieval computation from the LLM cost of the surrounding MIGRES framework.

8.10 Fairness Controls

The following should remain fixed across systems:

- dataset split;
- question set;
- document corpus;
- final retrieval depth k;
- MIGRES components outside retrieval;
- answer-generation model;
- stopping conditions;
- evaluation scripts;
- random seeds where applicable.

Only the retrieval method should change.

8.11 Reproducibility Configuration

The final experiment record should contain:

Parameter	**Value**
Dataset	Exact release and split
Corpus	Exact corpus version
First-stage retriever (FSR)	Selected empirically (Section 2); fixed for all systems thereafter
Reranker	None / MonoBERT / MonoT5 / DuoBERT / ColBERTv2 / LambdaMART / RankT5 / MMR / AIR / Dartboard-RIG / RankGPT / GAR
Baseline checkpoint	Exact model/checkpoint
GAR embedding model	`multi-qa-mpnet-base-cos-v1`
Final retrieval depth k	Fixed value for all systems
FSR candidate pool K_0	Validation-selected GAR value
α	Validation-selected value
β	Validation-selected value
γ	Validation-selected value
Bayesian optimization seed	Recorded value
FSR implementation	Exact library/index
Hardware	CPU/GPU configuration
Software	Python and package versions
MIGRES configuration	Exact prompts/model/settings

9 Conclusion

This project proposes Gap-Aware Retrieval (GAR) as a reranking algorithm for iterative information-seeking systems, applied on top of a first-stage retriever's candidates. GAR is evaluated inside MIGRES rather than replacing the MIGRES framework itself.
The core idea is:

$$\boxed{\text{Relevant} + \text{Gap-covering} + \text{Non-redundant}}$$

implemented as:

$$\boxed{S(d \mid q, E_t, G_t) = \alpha R(d, q) + \beta C(d, G_t) + \gamma N(d, E_t)}$$

The complete pipeline is:

First-Stage Retrieval → Sentence Transformer → Cosine Similarity
→ GAR Scoring → Top-k → MIGRES

GAR has no training stage in the primary experiment. Its scoring hyperparameters are selected on D_{val} through the complete MIGRES+FSR+GAR pipeline using Bayesian optimization and then frozen for D_{test}.
The central empirical question is whether reranking with the information gap and previously collected evidence produces a more useful ranking than standard reranking algorithms applied to the same FSR candidates.

References

Keheng Wang, Feiyu Duan, Peiguang Li, Sirui Wang, and Xunliang Cai. LLMs know what they need: Leveraging a missing information guided framework to empower retrieval-augmented generation. In *Proceedings of the 31st International Conference on Computational Linguistics*, pages 2379–2400. Association for Computational Linguistics, 2025. URL https://aclanthology.org/2025.coling-main.163/.

Vikas Yadav, Steven Bethard, and Mihai Surdeanu. Unsupervised alignment-based iterative evidence retrieval for multi-hop question answering. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 4514–4525. Association for Computational Linguistics, 2020. doi: 10.18653/v1/2020.acl-main.414. URL https://aclanthology.org/2020.acl-main.414/.

Marc Pickett, Jeremy Hartman, Ayan Kumar Bhowmick, Raquib-ul Alam, and Aditya Vempaty. Dartboard: Better RAG using relevant information gain. *arXiv preprint arXiv:2407.12101*, 2024. URL https://arxiv.org/abs/2407.12101.

Nils Reimers and Iryna Gurevych. Sentence-BERT: Sentence embeddings using siamese BERT-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing*, pages 3982–3992. Association for Computational Linguistics, 2019. doi: 10.18653/v1/D19-1410. URL https://aclanthology.org/D19-1410/.

Jeff Johnson, Matthijs Douze, and Hervé Jégou. Billion-scale similarity search with gpus. *arXiv preprint arXiv:1702.08734*, 2017. URL https://arxiv.org/abs/1702.08734.

Stephen Robertson and Hugo Zaragoza. The probabilistic relevance framework: Bm25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4):333–389, 2009. doi: 10.1561/1500000019. URL https://doi.org/10.1561/1500000019.

Vladimir Karpukhin, Barlas Oguz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. Dense passage retrieval for open-domain question answering. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing*, pages 6769–6781. Association for Computational Linguistics, 2020. doi: 10.18653/v1/2020.emnlp-main.550. URL https://aclanthology.org/2020.emnlp-main.550/.

Gautier Izacard, Mathilde Caron, Lucas Hosseini, Sebastian Riedel, Piotr Bojanowski, Armand Joulin, and Edouard Grave. Unsupervised dense information retrieval with contrastive learning. In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, pages 2665–2679. Association for Computational Linguistics, 2022. URL https://aclanthology.org/2022.emnlp-main.738/.

Keshav Santhanam, Omar Khattab, Jon Saad-Falcon, Christopher Potts, and Matei Zaharia. ColBERTv2: Effective and efficient retrieval via lightweight late interaction. In *Proceedings of the 2022 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 3715–3734. Association for Computational Linguistics, 2022. doi: 10.18653/v1/2022.naacl-main.272. URL https://aclanthology.org/2022.naacl-main.272/.

Thibault Formal, Carlos Lassance, Benjamin Piwowarski, and Stéphane Clinchant. SPLADE: Sparse lexical and expansion model for first stage ranking. *arXiv preprint arXiv:2107.05720*, 2021. URL https://arxiv.org/abs/2107.05720.

Rodrigo Nogueira and Kyunghyun Cho. Passage re-ranking with BERT. *arXiv preprint arXiv:1901.04085*, 2019. URL https://arxiv.org/abs/1901.04085.

Rodrigo Nogueira, Wei Yang, Kyunghyun Cho, and Jimmy Lin. Multi-stage document ranking with BERT. *arXiv preprint arXiv:1910.14424*, 2019. URL https://arxiv.org/abs/1910.14424.

Rodrigo Nogueira, Zhiying Jiang, Ronak Pradeep, and Jimmy Lin. Document ranking with a pre-trained sequence-to-sequence model. In *Findings of the Association for Computational Linguistics: EMNLP 2020*, pages 708–718. Association for Computational Linguistics, 2020. doi: 10.18653/v1/2020.findings-emnlp.63. URL https://aclanthology.org/2020.findings-emnlp.63/.

Christopher J. C. Burges. From RankNet to LambdaRank to LambdaMART: An overview. Technical Report MSR-TR-2010-82, Microsoft Research, 2010. URL https://www.microsoft.com/en-us/research/publication/from-ranknet-to-lambdarank-to-lambdamart-an-overview/.

Jaime Carbonell and Jade Goldstein. The use of MMR, diversity-based reranking for reordering documents and producing summaries. In *Proceedings of the 21st Annual International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 335–336. ACM, 1998. doi: 10.1145/290941.291025. URL https://doi.org/10.1145/290941.291025.

Honglei Zhuang, Zhen Qin, Rolf Jagerman, Kai Hui, Ji Ma, Jing Lu, Jianmo Ni, Xuanhui Wang, and Michael Bendersky. RankT5: Fine-tuning T5 for text ranking with ranking losses. In *Proceedings of the 46th International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 2308–2313. ACM, 2023. doi: 10.1145/3539618.3592047. URL https://doi.org/10.1145/3539618.3592047.

Weiwei Sun, Lingyong Yan, Xinyu Ma, Shuaiqiang Wang, Pengjie Ren, Zhumin Chen, Dawei Yin, and Zhaochun Ren. Is ChatGPT good at search? investigating large language models as re-ranking agents. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 14918–14937. Association for Computational Linguistics, 2023. URL https://aclanthology.org/2023.emnlp-main.923/.

Sentence Transformers. multi-qa-mpnet-base-cos-v1. Hugging Face model card, 2020. URL https://huggingface.co/sentence-transformers/multi-qa-mpnet-base-cos-v1. 768-dimensional sentence-transformers model designed for semantic search.

Jasper Snoek, Hugo Larochelle, and Ryan P. Adams. Practical bayesian optimization of machine learning algorithms. In *Advances in Neural Information Processing Systems*, volume 25, 2012. URL https://arxiv.org/abs/1206.2944.

Johannes Welbl, Pontus Stenetorp, and Sebastian Riedel. Constructing datasets for multi-hop reading comprehension across documents. *Transactions of the Association for Computational Linguistics*, 6:287–302, 2018. doi: 10.1162/tacl_a_00021. URL https://aclanthology.org/Q18-1021/.

Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William Cohen, Ruslan Salakhutdinov, and Christopher D. Manning. Hotpotqa: A dataset for diverse, explainable multi-hop question answering. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*,

pages 2369–2380. Association for Computational Linguistics, 2018. doi: 10.18653/v1/D18-1259. URL https://aclanthology.org/D18-1259/.

Harsh Trivedi, Niranjan Balasubramanian, Tushar Khot, and Ashish Sabharwal. Musique: Multihop questions via single-hop question composition. *Transactions of the Association for Computational Linguistics*, 10:539–554, 2022. doi: 10.1162/tacl_a_00475. URL https://aclanthology.org/2022.tacl-1.31/.

Tom Kwiatkowski, Jennimaria Palomaki, Olivia Redfield, Michael Collins, Ankur Parikh, Chris Alberti, Danielle Epstein, Illia Polosukhin, Jacob Devlin, Kenton Lee, Kristina Toutanova, Llion Jones, Matthew Kelcey, Ming-Wei Chang, Andrew M. Dai, Jakob Uszkoreit, Quoc Le, and Slav Petrov. Natural questions: A benchmark for question answering research. *Transactions of the Association for Computational Linguistics*, 7:452–466, 2019. doi: 10.1162/tacl_a_00276. URL https://aclanthology.org/Q19-1026/.

Mandar Joshi, Eunsol Choi, Daniel Weld, and Luke Zettlemoyer. Triviaqa: A large scale distantly supervised challenge dataset for reading comprehension. In *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics*, pages 1601–1611. Association for Computational Linguistics, 2017. doi: 10.18653/v1/P17-1147. URL https://aclanthology.org/P17-1147/.

Mor Geva, Daniel Khashabi, Elad Segal, Tushar Khot, Dan Roth, and Jonathan Berant. Did aristotle use a laptop? a question answering benchmark with implicit reasoning strategies. *Transactions of the Association for Computational Linguistics*, 9:346–361, 2021. doi: 10.1162/tacl_a_00370. URL https://aclanthology.org/2021.tacl-1.21/.

Kalervo Järvelin and Jaana Kekäläinen. Cumulated gain-based evaluation of ir techniques. *ACM Transactions on Information Systems*, 20(4):422–446, 2002. doi: 10.1145/582415.582418. URL https://doi.org/10.1145/582415.582418.

Chris Burges, Tal Shaked, Erin Renshaw, Ari Lazier, Matt Deeds, Nicole Hamilton, and Greg Hullender. Learning to rank using gradient descent. In *Proceedings of the 22nd International Conference on Machine Learning*, pages 89–96. ACM, 2005. doi: 10.1145/1102351.1102363.